

Deep Learning for BTC/USDT Forecasting

Yu ByeongHyeon (20211199)

Step 1. Problem Definition

Research Question

Given recent BTC/USDT OHLCV sequences, can a deep learning model extract a **directional signal** that is usable for trading decisions (long/short) without external data?

- **Context:** Crypto runs 24/7, fewer market closures/gaps; consistent hourly structure.
- **Inputs:** Price/volume-derived features (OHLCV, log-returns, common TA indicators, time-of-day/week cycles).
- **Outputs:** Future return over a horizon; $\text{sign}(\hat{y})$ interpreted as directional signal.
- **Why DL:** Sequential dependencies/regime shifts are hard to hand-engineer; RNN-family models capture temporal dynamics.
- **Success Criteria:** Sound pipeline (no leakage) $\rightarrow$ stable training $\rightarrow$ interpretable, directionally consistent signals.

Design Principles (for Step 1)

- **Leakage-free** chronology: compute indicators/scaler on **train only**; build windows **after** splitting.
- **Simple baseline first**: single-horizon label, compact RNN head, standard optimizer; keep knobs small.
- **Diagnostics-first**: prioritize direction accuracy (with dead-zone) and stability across market regimes.

Step 2: Dataset Summary

Raw Data Source

The dataset consists of BTC/USDT 1-hour candles collected from the Binance public API, covering **October 15 2023 – October 15 2025** (two full years), totaling about **17 520 hourly observations**.

Features

- **Primary:** OHLCV – Open, High, Low, Close, Volume
- **Derived:** log-returns $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$, EMA(12, 26), RSI(14), MACD and signal, ATR(14)
- **Cyclical time features:** sine / cosine of hour-of-day and day-of-week

Dataset Scale and Structure

After removing NaNs and creating sliding windows of 240 hours (10 days of context), the dataset contains approximately **13 000 train**, **1 500 validation**, and **1 500 test** samples. This makes it a **medium-scale sequential dataset** with about 25 features per time step.

Preprocessing Skeleton (for Reproducibility)

```
1 df = df.loc[:, ~df.columns.duplicated()]
2 df = normalize_time_column(df)
3 df = df.dropna(subset=["open", "high", "low", "close", "volume", "log_return"])
4 import ta
5 df["ema_12"] = ta.trend.ema_indicator(df["close"], 12)
6 df["ema_26"] = ta.trend.ema_indicator(df["close"], 26)
7 df["rsi_14"] = ta.momentum.rsi(df["close"], 14)
8 _macd = ta.trend.MACD(df["close"])
9 df["macd"] = _macd.macd()
10 df["macd_sig"] = _macd.macd_signal()
11 df["atr_14"] = ta.volatility.average_true_range(df["high"], df["low"], df["close"], 14)
12 df = df.dropna().reset_index(drop=True)
```

Train/Val/Test Split & Windowing

```
1 # chronological split (with fallback)
2 scaler = StandardScaler().fit(df_train[feature_cols].values)
3
4 def make_xy_multi(df_split, window):
5     Xf = scaler.transform(df_split[feature_cols].values)
6     Yf = df_split[["y_2h", "y_4h", "y_6h"]].values
7     xs, ys = [], []
8     for i in range(window, len(Xf)):
9         xs.append(Xf[i-window:i, :])
10        ys.append(Yf[i])
11    return np.array(xs, np.float32), np.array(ys, np.float32)
```

Step 3. Appropriate Algorithms

- **Choice: GRU/LSTM** as first candidates — strong inductive bias for sequences, parameter-efficient, easier to stabilize with clipping/early stop, well-suited for moderate-size time series.
- **Key Hypothesis:** If performance is weak at 1h, the bottleneck is likely **label SNR**, not model capacity.
- **Plan:** Keep the training stack lean (Huber/Adam/clip) and **reformulate the label horizon** before adding model complexity.

Step 4. Experiments & Results — Structure

- **Exp A:** 1h target, MSE, window $L = 168$, single-output GRU/LSTM.
- **Exp B:** Label smoothing (4h cumulative), $L = 240$, time-cyclic features, Huber loss.
- **Exp C:** Single-output targets for $\{2h, 4h, 6h\}$ to benchmark stability vs timeliness.
- **Exp D:** Add TCN and Transformer under the same pipeline to test model-family effects.
- **Exp E:** Multi-output head ($2h, 4h, 6h$) for one-shot inference and direct comparison.

Exp A. 1h + MSE + L=168 (Baseline)

Outcome (observation)

Single-output GRU/LSTM trained with MSE on 1h return produced unstable directional signals around chance.

Issue / Likely Cause

Microstructure noise and bid-ask bounce dominate at 1h; low SNR overwhelms price-only features.

Design → Exp B

Increase horizon to **aggregate** noise; extend context; switch to **Huber** to reduce outlier impact.

Exp B. 4h Cumulative + L=240 + Huber

Design

Target $y^{(4h)} = \sum_{i=1}^4 r_{t+i}$; window $L = 240$; hour/day cyclic features; Huber loss + grad clip + early stop.

Outcome (observation)

Directional signals become more consistent; smaller variance across validation epochs; improved interpretability.

Issue / Next Gap

We only tested one longer horizon (4h). Need to map stability across horizons to find the sweet spot.

Design → Exp C

Single-output regressors for $\{2h, 4h, 6h\}$ with same pipeline to benchmark horizon sensitivity.

(Exp B) Label Smoothing & Training (Code)

```
1 def future_sum(series, k):
2     s = series.reset_index(drop=True)
3     return s.shift(-1).rolling(k).sum()
4
5 # 4h target and RNN head
6 y4 = future_sum(df["log_return"], 4).rename("y4h").to_frame().dropna()
7 model_4h = RNNMulti(in_feats, hidden=64, layers=2, kind="gru",
8                     dropout=0.2, out_dim=1, bidir=False)
9 crit = nn.HuberLoss(delta=1.0)
```

Exp C. Single-Output 2h/4h/6h (Stability Sweep)

Design

Train three separate single-output models for 2h, 4h, 6h using the same preprocessing and optimization.

Outcome (observation)

2h remains noisy; 4h shows a clear lift; 6h becomes the most stable in sign with lower variance across runs.

Issue / Next Gap

Which model family best leverages the smoothed labels? Are RNNs strictly better, or do CNN/Attention add value?

Design → Exp D

Introduce **TCN** and **Transformer** for a fair, pipeline-consistent comparison.

(Exp C) Windowing Per Horizon (Code)

```
1 def make_xy_single(df_split, target_col, window):
2     Xf = scaler.transform(df_split[feature_cols].values)
3     Yf = df_split[[target_col]].values
4     xs, ys = [], []
5     for i in range(window, len(Xf)):
6         xs.append(Xf[i-window:i, :])
7         ys.append(Yf[i])
8     return np.array(xs, np.float32), np.array(ys, np.float32)
```

Exp D. Add TCN and Transformer (Model Diversity)

Design

Keep labels and preprocessing fixed; compare GRU/LSTM vs TCN (dilated causal) vs Transformer (encoder-only lite).

Outcome (observation)

GRU/LSTM remain robust on this dataset size; Transformer is competitive but not dominant; TCN underperforms in stability.

Issue / Next Gap

Multiple single-output models are cumbersome for inference and comparison; need unified outputs.

Design → Exp E

Use **multi-output**($2h, 4h, 6h$) forward pass yields all horizons for direct, consistent comparison.

(Exp D) Model Definitions (Code)

```
1 class TCNMulti(nn.Module):
2     def __init__(self, in_feats, ch=64, depth=4, k=3, dropout=0.2, out_dim=3):
3         super().__init__()
4         self.blocks = nn.ModuleList([
5             TCNBlock(in_feats if i==0 else ch, ch, k=k, dilation=2**i, dropout=dropout)
6             for i in range(depth)])
7         self.pool = nn.AdaptiveAvgPool1d(1)
8         self.head = nn.Sequential(nn.Flatten(), nn.Linear(ch, ch), nn.ReLU(),
9                                     nn.Dropout(dropout), nn.Linear(ch, out_dim))
10    def forward(self, x): # x: (B,W,F)
11        x = rearrange(x, 'b w f -> b f w')
12        for blk in self.blocks: x = blk(x)
13        x = self.pool(x)
14        return self.head(x)
15
16 class TransEncMulti(nn.Module):
17     def __init__(self, in_feats, d_model=64, nhead=4, num_layers=2, dim_ff=128, dropout=0.1,
18         out_dim=3):
19         super().__init__()
```

(Exp D) Transformer Encoder (cont.)

```
1      self.inproj = nn.Linear(in_feats, d_model)
2      layer = nn.TransformerEncoderLayer(
3          d_model, nhead, dim_ff, dropout, batch_first=True
4      )
5      self.enc = nn.TransformerEncoder(layer, num_layers)
6      self.head = nn.Sequential(
7          nn.Linear(d_model, d_model), nn.ReLU(),
8          nn.Dropout(dropout), nn.Linear(d_model, out_dim)
9      )
10
11  def forward(self, x):                                # x: (B,W,F)
12      z = self.inproj(x)
13      z = self.enc(z)
14      return self.head(z[:, -1, :])
```

Exp E. Multi-Output Head (2h/4h/6h)

Design

Shared temporal encoder with a head producing (y^{2h}, y^{4h}, y^{6h}) simultaneously.

Outcome (observation)

Inference becomes one-shot; comparison across horizons is consistent; 4h/6h signals remain strong and stable.

Why it helps

Shared representation learns regime features once; reduces variance between separately trained heads; simpler evaluation.

(Exp E) RNN Multi-Output Training (Code)

```
1 class RNNMulti(nn.Module):
2     def __init__(self, in_feats, hidden=64, layers=2, kind="gru", dropout=0.2, out_dim=3,
3         bidir=False):
4         super().__init__()
5         rnn_cls = nn.GRU if kind=="gru" else nn.LSTM
6         self.rnn = rnn_cls(in_feats, hidden, num_layers=layers,
7             batch_first=True, dropout=(dropout if layers>1 else 0),
8             bidirectional=bidir)
9         last_dim = hidden * (2 if bidir else 1)
10        self.head = nn.Sequential(nn.Linear(last_dim, last_dim), nn.ReLU(),
11            nn.Dropout(dropout), nn.Linear(last_dim, out_dim))
12
13        self.kind = kind
14    def forward(self, x): # x: (B,W,F)
15        out = self.rnn(x)[0]
16        return self.head(out[:, -1, :])
17
18    def train_one(model, epochs=25, lr=1e-3, patience=6, name="MODEL"):
19        model = model.to(device)
20        opt = torch.optim.Adam(model.parameters(), lr=lr)
```

(Exp E) Training Loop (cont.)

```
1 best_loss, wait = 1e9, 0
2 for ep in range(1, epochs+1):
3     model.train()
4     for xb, yb in train_dl:
5         xb, yb = xb.to(device), yb.to(device)
6         opt.zero_grad()
7         pred = model(xb)
8         loss = crit(pred, yb)
9         loss.backward()
10        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
11        opt.step()
12    val_loss, _, _, _ = evaluate(model, val_dl, crit)
13    if val_loss < best_loss - 1e-9:
14        best_loss, wait = val_loss, 0
15        torch.save(model.state_dict(), f"best_{name}.pt")
16    else:
17        wait += 1
18        if wait >= patience: break
19
```

- **Metrics:** MAE, RMSE, Direction Accuracy with a small dead-zone $|\hat{y}| < 10^{-4}$.
- **Regime Analysis:** Split test by realized volatility (rolling 24h std; top 30% = High) and compare per-horizon metrics.
- **Simple Backtest:** Long if $\hat{y} > 0$, short if $\hat{y} < 0$; 1 unit; no fees/slippage; summarize by horizon.

Evaluation Helpers (Code)

```
1 def dir_acc(y_true, y_pred, eps=1e-4):
2     mask = (np.abs(y_pred) >= eps)
3     return ((y_true[mask] >= 0) == (y_pred[mask] >= 0)).mean() if mask.any() else np.nan
4
5 def metrics_multi(y_true, y_pred):
6     out = {}
7     names = ["2h", "4h", "6h"]
8     for i, name in enumerate(names):
9         yt, yp = y_true[:,i], y_pred[:,i]
10        out[f"{name}_MAE"] = mean_absolute_error(yt, yp)
11        out[f"{name}_RMSE"] = np.sqrt(mean_squared_error(yt, yp))
12        out[f"{name}_DA"] = dir_acc(yt, yp, eps=0.0)
13        out[f"{name}_DAe"] = dir_acc(yt, yp, eps=1e-4)
14    return out
```

Test Performance Summary (Test Split)

Model	2h MAE	2h RMSE	2h DA(%)	4h MAE	4h RMSE	4h DA(%)	6h MAE	6h RMSE	6h DA(%)
GRU	0.0034	0.0050	49.17	0.0036	0.0052	72.50	0.0037	0.0053	76.67
LSTM	0.0034	0.0050	51.19	0.0036	0.0052	71.67	0.0038	0.0054	75.48
TCN	0.0036	0.0051	47.62	0.0052	0.0075	49.88	0.0077	0.0106	50.83
TRANS	0.0034	0.0050	50.83	0.0043	0.0061	66.90	0.0056	0.0081	60.12

All numbers are from the held-out test split; DA = direction accuracy.

Key Findings from Test Split

- **Signal horizon matters:** 2h $\approx$ coin-flip, while **4h/6h** show strong lift (GRU/LSTM: 72–77% DA).
- **Model family:** GRU/LSTM > TRANS > TCN on this medium-scale dataset and feature set.
- **Bias-variance trade-off:** RNNs' inductive bias helps with limited data; Transformer is competitive at 4h but not dominant.

Backtest (GRU Sample on Test Split)

One-unit long/short, no fees; horizons = 2h/4h/6h

Horizon	RET	SR	MDD	Hit	Trades
2h	-0.0713	-0.0172	-0.1711	0.4917	190
4h	2.7800	0.5204	-0.0294	0.7250	242
6h	3.8741	0.5995	-0.0307	0.7667	148

RET: cumulative return (unitless), SR: Sharpe-like ratio (per-run scale), MDD: max drawdown, Hit: accuracy over trades.

Backtest Summary (All Models, Test Split)

Metric	GRU	LSTM	TCN	TRANS
2h_Hit	0.491667	0.511905	0.486905	0.508333
2h_MDD	-0.171061	-0.143035	-0.270464	-0.286521
2h_RET	-0.071281	-0.023538	-0.017604	0.075178
2h_SR	-0.017157	-0.005665	-0.004237	0.018096
2h_Trades	190	262	3	11
4h_Hit	0.725000	0.716667	0.501190	0.669048
4h_MDD	-0.029371	-0.029371	-0.525113	-0.032604
4h_RET	2.780027	2.708835	-0.014469	2.144349
4h_SR	0.520435	0.503666	-0.002402	0.381051
4h_Trades	242	242	7	388
6h_Hit	0.766667	0.754762	0.508333	0.601190
6h_MDD	-0.030652	-0.037530	-0.767682	-0.062580
6h_RET	3.874070	3.921944	-0.156829	2.086444
6h_SR	0.599478	0.609620	-0.020816	0.288147
6h_Trades	148	172	1	256

Best-performing Setup

Final Configuration

- **Model:** GRU/LSTM encoder + multi-output head ($2h, 4h, 6h$)
- **Target:** 4h/6h cumulative log-return labels
- **Window:** 240 hours (10 days)
- **Training:** Huber loss + Adam optimizer + gradient clipping
- **Features:** OHLCV, log-returns, EMA(12/26), RSI(14), MACD, ATR(14), time-of-day/week sin-cos

Why It Worked & Data Insights

Reasons for Success

- **Noise reduction:** Longer horizons (4h/6h) average out microstructure noise and random fluctuations.
- **Efficient inductive bias:** RNNs (GRU/LSTM) capture mid-term temporal dependencies better than CNN/Transformer on medium data.
- **Shared head:** Multi-output architecture ensures consistency across horizons and prevents overfitting to one label.

Interpretation

- 4h/6h signals are **statistically and practically meaningful**.
- Confirms that deep sequence models can extract tradable structure even from simple OHLCV data when leakage is controlled.

Conclusions & Insights

Why 1h underperformed

- **Low SNR at 1h:** microstructure noise, bid-ask bounce, mean-reversion at very short horizons.
- **Volatility clustering:** direction flips frequently at 1h; small true moves drowned by noise.
- **Price-only limitation:** without orderbook/news, ultra-short horizons are hard to forecast robustly.

What worked best and why

- **Label reformulation** (4h/6h cumulative) increased predictability by averaging noise.
- **GRU/LSTM** performed best overall: parameter-efficient, strong temporal bias for this data scale.
- **Multi-output head** enabled consistent, comparable signals across horizons in a single pass.

Limitations & Next Steps

Limitations

Single asset; simple simulation (no fees/funding/slippage); lightweight models; no walk-forward updates yet.

Next Steps

Now that simple backtesting validates the signals, the plan is to **connect directly to the Binance API** and run real-time trading simulations and live backtesting with execution constraints (fees, slippage, rate limits).

Thank you!

Questions & discussion are welcome.